

Sistemas Distribuídos (DCC/UFRJ)

Módulo 6 - Parte 1 - Aula 2: Relógios lógicos

Profa. Silvana Rossetto

Para muitas aplicações, a **ordem relativa dos eventos** é mais importante que o tempo físico exato no qual eles ocorreram

- *ex., comando make*

Considerando **evento** a ocorrência de uma ação única de **comunicação** ou de **transição de estado**

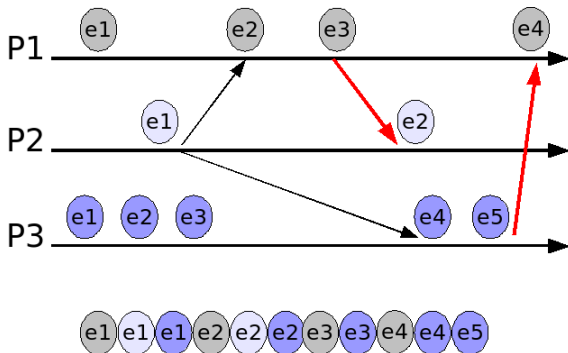
QUESTÃO 1: Como ordenar os eventos dentro de um processo individual?

A sequência de eventos de um processo individual $p(i)$ pode ser ordenada de forma única, usando a seguinte representação:

- $e^a \rightarrow e^b$, sss o evento e^a ocorreu antes do evento e^b em $p(i)$
- o histórico de execução do processo $p(i)$ é a série de eventos: $h(i) = e(i)^1, e(i)^2, \dots e(i)^n$

QUESTÃO 2: Como ordenar os eventos de um sistema distribuído?

Uma execução distribuída consiste de um **conjunto de eventos** sequenciais em cada processo



Dificuldade para estabelecer uma ordem total correta sobre todos os eventos

Causalidade em eventos distribuídos

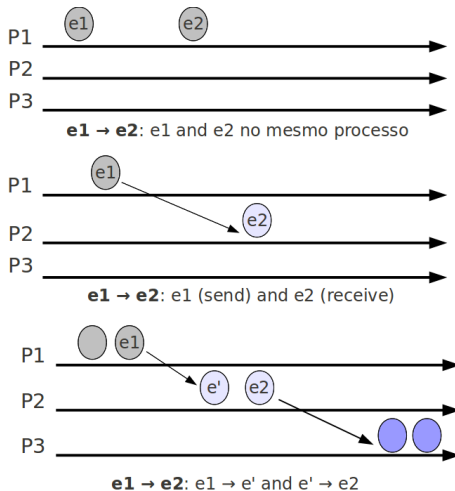
Para alguns algoritmos distribuídos, a informação exata sobre uma execução do sistema não fornece muitas informações sobre outras possíveis execuções do sistema

Mas, se o evento A ocorreu antes do evento B, então é possível que o **evento A afete o evento B** por **causalidade** ($A \rightarrow B$)

Assim, é possível estabelecer a **ordem relativa de ocorrência dos eventos** considerando todos os processos da aplicação

Causalidade em eventos distribuídos

- $a \rightarrow b$: **a** ocorre antes de **b**
 - **a** e **b** são consecutivos dentro do processo $p(i)$ e **a** terminou antes de **b** começar
 - **a** = **send(m)** no processo P e **b** = **receive(m)** no processo Q
 - **a** e **b** estão relacionados por transitividade
- $a || b$: os eventos **a** e **b** são concorrentes

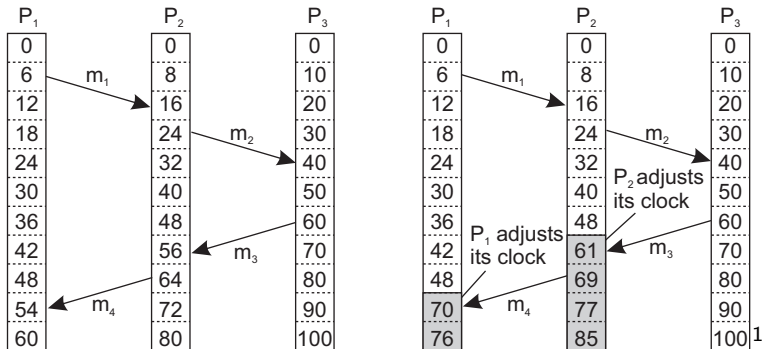


Lamport propôs o conceito de **relógio lógico**, consistente com as relações de **causalidade de eventos**

ALGORITMO

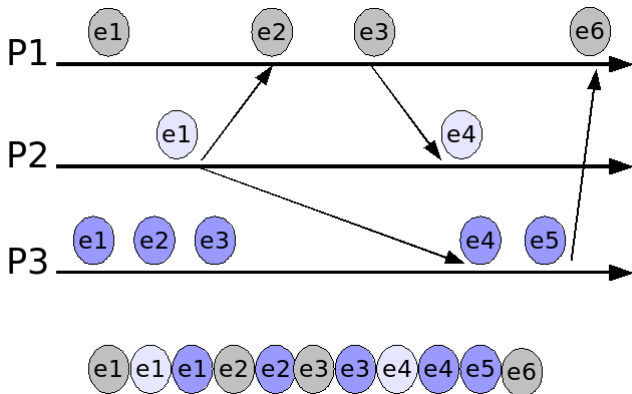
- cada processo $p(i)$ tem um contador local $LT(p)$ com valor inicial 0
- para cada evento executado, exceto *receive*, o contador é incrementado de 1:
 $LT(p) = LT(p) + 1$
- ao enviar uma mensagem, anexa o valor do contador LT à mensagem: $LT(m) = LT(p)$
- ao receber uma mensagem, atualiza o valor de LT local: $LT(p) = \max(LT(m), LT(p)) + 1$

Relógios de Lamport



¹Fonte: Steen-Tanenbaum, distributed-systems.net.

Permite estabelecer uma ordem parcial correta sobre todos os eventos



Consistência com causalidade

- se $a \rightarrow b$, então $LT(a) < LT(b)$ (independente de a e b ocorrerem no mesmo processo)
- se $a || b$, então $LT(a)$ e $LT(b)$ podem ter valores arbitrários

O protocolo pode ser estendido para impor uma ordem total (arbitrária) sobre todos os eventos adicionando o identificador do processo ao LT no caso de empates (decorrentes de eventos concorrentes)

M. van Steen and A.S. Tanenbaum, “Distributed Systems”, 3rd ed., www.distributed-systems.net, 2017

- **seção 6.2 (pag 310-313)**